

Natural Language Processing (NLP)

Large Language Models Decoding

LLMS

By:

Dr. Zohair Ahmed



 www.youtube.com/@ZohairAI 

 www.begindiscovery.com

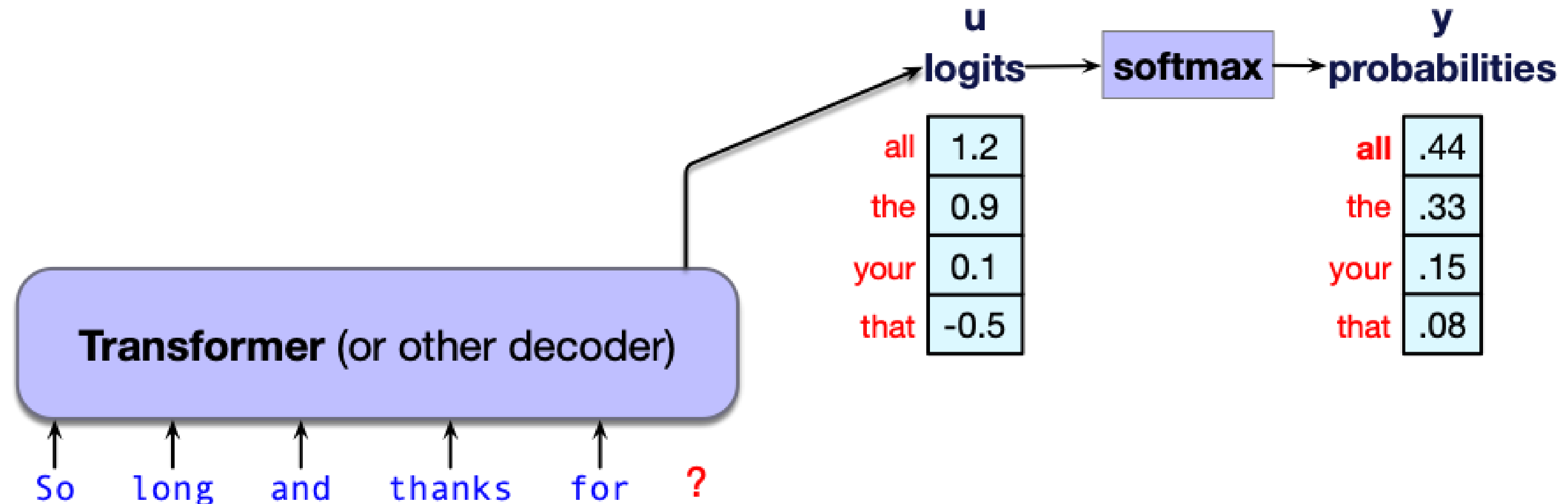
Large
Language
Models

Decoding Strategies

Where does token probability come from?

- The internal networks for LLMs generate real-valued scores called **logits** for each token in the vocabulary.
- Score vector **u** of shape $[1 \times |V|]$ is turned into a probability by softmax

$$\mathbf{y} = \text{softmax}(\mathbf{u})$$



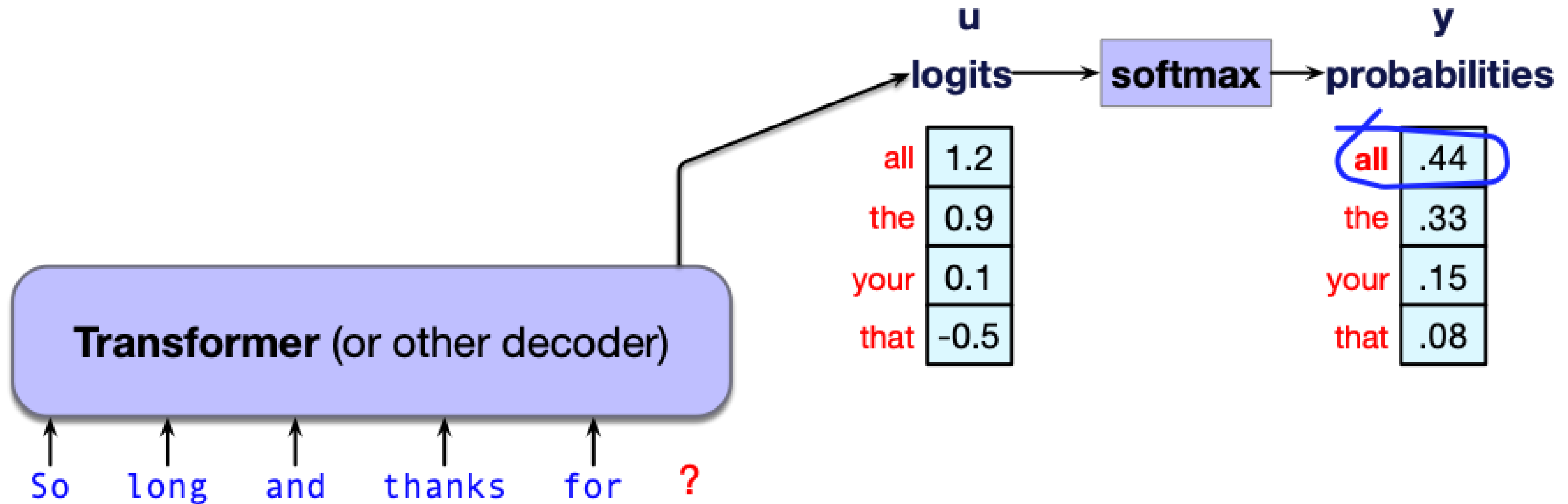
Decoding

- After the model computes **probabilities for each next token** (using logits $\rightarrow$ softmax), it must **choose one token** to output.
- This choosing step is called **decoding**.
- Think of it like this:
- The model gives you a list of probabilities for all tokens
- You must pick one token to generate
- The strategy you use to pick the token = **decoding method**
- **Examples of decoding methods**
 - Greedy decoding
 - Sampling
 - Top-k sampling
 - Top-p (nucleus) sampling
 - Beam search
 - But for now, just remember:
 - **Decoding = choosing the next word according to the model's probabilities.**

Greedy Decoding

- A **greedy algorithm** always chooses the **best immediate option** at each step even if that choice may not lead to the best long-term result.
- In language models:
- **Greedy decoding = always pick the token with the highest probability.**
- It does NOT consider future possibilities.
- It may miss better sentences overall.
- **Mathematical Formula**
 - $\hat{w}_i = \operatorname{argmax}_{w \in V} P(w \mid w_{<i})$
 - $\hat{w}_i$ = the chosen next token
 - V = the vocabulary
- $P(w \mid w_{<i})$ = the probability of token **w** given all previous tokens
- **argmax** = choose the token **w** that has the **largest probability**
- So the model does:
 - Look at all possible next tokens
 - Compute probability for each
 - Pick the **highest-probability** one
 - Add it to the output
 - Repeat
- If two or more tokens have exactly the same highest probability, model will choose the first one in its internal vocabulary order.

Greedy decoding: choosing "all"



$$\hat{w}_t = \operatorname{argmax}_{w \in V} P(w|w_{<t})$$

We don't use greedy decoding

Reason 1: Predictability → Repetition

- Because greedy decoding is predictable:
 - It often gets stuck in **loops**
 - It often repeats phrases
 - It produces boring patterns like:
“I think the best way to do this is to think about the best way to do this...”
- Why?
 - Because the highest-probability token is often a **common**, **safe**, and **generic** word.
 - And if that word is always chosen, the model keeps spiraling into repetitive patterns.

Reason 2: Deterministic Behavior

- Greedy decoding is **deterministic**.
- That means:
 - Same input → same output
 - Every time
 - No variation
 - No creativity
- This is why greedy decoding is **too predictable** for generating natural text like stories, answers, or conversation.

Reason 3: It Ignores Future Possibilities

- Greedy decoding only cares about **this immediate step**, not long-term sentence quality.

- Imagine writing a story where you always choose the next word based **ONLY** on the strongest short-term guess.
You would never explore interesting or creative wording because you always pick “safe” words.

- **People Prefer TEXT WITH VARIETY → so they use Sampling**
- To get more natural, human-like text, we introduce **randomness**.
- **Greedy decoding = always pick the highest-probability token**
 - → deterministic
 - → predictable
 - → generic
 - → repetitive
- **Sampling = pick according to probability distribution**
 - → diverse
 - → creative
 - → more natural



We don't use greedy decoding

Reason 1: Predictability → Repetition

- Because greedy decoding is predictable:
 - It often gets stuck in **loops**
 - It often repeats phrases
 - It produces boring patterns like:
“I think the best way to do this is to think about the best way to do this...”
- Why?
 - Because the highest-probability token is often a **common**, **safe**, and **generic** word.
 - And if that word is always chosen, the model keeps spiraling into repetitive patterns.
- Greedy decoding only cares about **this immediate step**, not long-term sentence quality.
 - Imagine writing a story where you always choose the next word based ONLY on the strongest short-term guess.
You would never explore interesting or creative wording because you always pick “safe” words.
- **People Prefer TEXT WITH VARIETY → so they use Sampling**
- To get more natural, human-like text, we introduce **randomness**.
- **Greedy decoding = always pick the highest-probability token**

Reason 2: Deterministic Behavior

- Greedy decoding is **deterministic**.
 - Same input → same output
 - Every time
 - No variation
 - No creativity
- This is why greedy decoding is **too predictable** for generating natural text like stories, answers, or conversation.
- → deterministic
- → predictable
- → generic
- → repetitive
- **Sampling = pick according to probability distribution**
 - → diverse
 - → creative
 - → more natural

Reason 3: It Ignores Future Possibilities



Random Sampling

- **Sampling = pick according to the probability distribution**
- It means:
- Tokens with **higher probability** are *more likely*
- Tokens with **lower probability** are *less likely*
- BUT **any token** with non-zero probability *can be chosen*
- Not just the highest one.
- **Example to Make It Crystal Clear**
- Suppose the model gives probabilities for next token:
- **Token , Probability**
- “the” , 0.60
- “a” , 0.25
- “it” , 0.10
- “this” , 0.05

Greedy decoding:

- ALWAYS picks:
- “the”

Sampling:

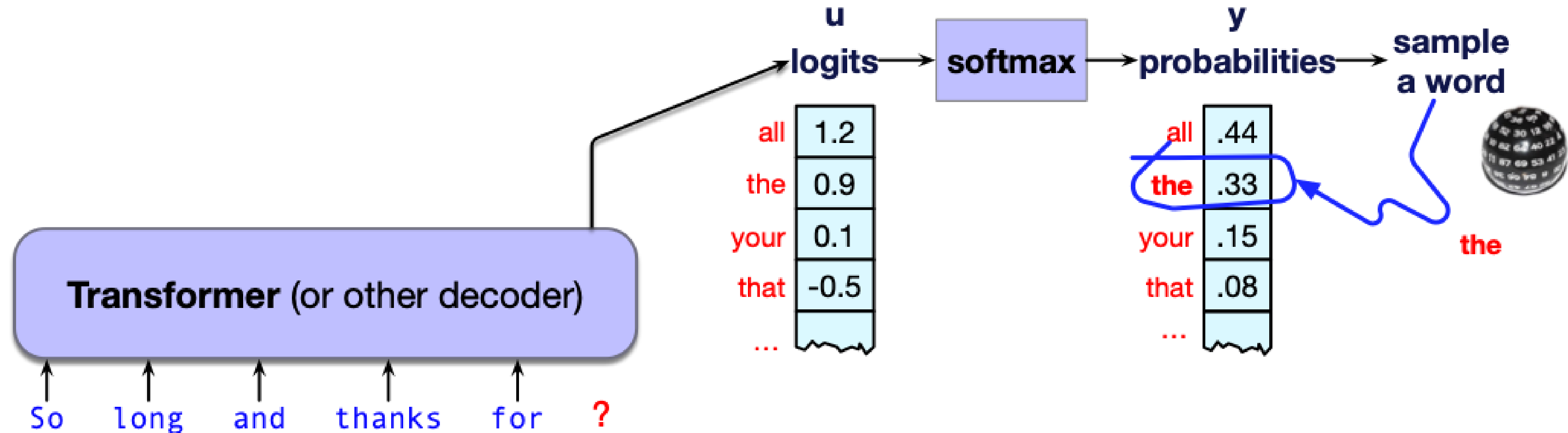
- You **roll a weighted dice**:
- “the” has a 60% chance
- “a” has a 25% chance
- “it” has a 10% chance
- “this” has a 5% chance

So sampling might pick:

- “the” (most of the time)
 - “a” (sometimes)
 - “it” (rarely)
 - “this” (very rarely)
- This adds **diversity** and **creativity**.



Random Sampling



Random Sampling for Text Generation

- This pseudocode shows how a language model generates text **one word at a time** by randomly picking the next word based on probabilities.
- **Step-by-step:**
 1. $i \leftarrow 1$
 2. $w_1 \sim p(w)$
 3. **while** $w_i \neq \text{EOS}$
 4. $i \leftarrow i + 1$
 5. $w_i \sim p(w_i \mid w_{<i})$
 1. Start at position 1 (first word to generate)
 2. Sample the first word from the overall word probability distribution
 3. Keep going until you hit the **End-Of-Sequence** token
 4. Move to the next position
 5. Sample the next word from the **conditional probability** given all previous words.

Random Sampling for Text Generation

- Unlike greedy decoding (which always picks the highest-probability word), random sampling **draws from the full distribution**.
 - So if "dog" has 30% probability and "cat" has 20%, either could be chosen with "dog" being more likely but not guaranteed.
 - It produces more diverse and creative text, but can sometimes generate incoherent output because low-probability (weird) words also have a chance of being picked.
- That's why techniques like **temperature scaling**, **top-k**, and **top-p (nucleus sampling)** were introduced they control *how random* the sampling is.



Factors in word sampling: quality and diversity

- Emphasize **high-probability** words
- + **quality**: more accurate, coherent, and factual,
- - **diversity**: boring, repetitive.
- Emphasize **middle-probability** words
- + **diversity**: more creative, diverse,
- - **quality**: less factual, incoherent



Temperature Sampling

- Reshape the probability distribution
- **Increase** the probability of the high probability tokens
- **Decrease** the probability of the low probability tokens
- Temperature controls **how sharp or flat** the probability distribution is by dividing logits by a parameter τ (**tau**) before softmax:
 - ~~$y = \text{softmax}(u)$~~
 - $y = \text{softmax}(u / \tau)$
- Think of temperature like confidence.
- **Low temperature** = the model is very confident and sticks to safe choices.
- **High temperature** = the model explores more freely and takes risks.
- **Example:** Say logits give "the" = 60%, "a" = 25%, "banana" = 15%
 - $\tau = 0.5 \rightarrow$ maybe "the" = 90%, "a" = 9%, "banana" = 1% (almost always picks "the")
 - $\tau = 2.0 \rightarrow$ maybe "the" = 40%, "a" = 32%, "banana" = 28% (any word could be picked)
- you're just **scaling the logits** before softmax, but the effect on generation quality is huge.



Temperature Sampling: Quick Example

- **Raw logits:** $u_1 = 2$, $u_2 = 1$
- **$\tau = 1$ (normal):**
 - $\text{softmax}(2, 1) \rightarrow e^2 = 7.39$, $e^1 = 2.72$
 - $7.39/10.11 = 73\%$, $2.72/10.11 = 27\%$
- **$\tau = 0.5$ (sharp):**
 - Divide logits: $2/0.5 = 4$, $1/0.5 = 2$
 - $\text{softmax}(4, 2) \rightarrow e^4 = 54.6$, $e^2 = 7.39$
 - $54.6/61.99 = 88\%$, $7.39/61.99 = 12\%$
- **$\tau = 2$ (flat):**
 - Divide logits: $2/2 = 1$, $1/2 = 0.5$
 - $\text{softmax}(1, 0.5) \rightarrow e^1 = 2.72$, $e^{0.5} = 1.65$
 - $2.72/4.37 = 62\%$, $1.65/4.37 = 38\%$
- Same logits, three different outcomes.
- Lower $\tau \rightarrow$ winner takes more.
- Higher $\tau \rightarrow$ more equal chances.

τ	Word A	Word B
0.5	88%	12%
1.0	73%	27%
2.0	62%	38%



Uniform : Temperature Sampling

- **Uniform** means all words get **roughly equal probability**. like in the example where 73% vs 27% became 62% vs 38%.
- If you have 4 words, a perfectly uniform distribution would be **25%, 25%, 25%, 25%** every word has the same chance.
- So when we say high temperature makes probabilities "more uniform," it means the gap between high-prob and low-prob words **shrinks**
- They start becoming closer to each other,



Temperature Sampling: τ value

τ value	Effect	Behavior
$\tau < 1$ (e.g., 0.2)	Sharpens distribution	High-prob words get higher , low-prob words get lower → more focused, repetitive output
$\tau = 1$	No change	Original distribution, normal random sampling
$\tau > 1$ (e.g., 2.0)	Flattens distribution	Probabilities become more uniform → more random, creative, but riskier output
$\tau \rightarrow 0$	Extremely sharp	Behaves like greedy decoding (always picks the top word)



Temperature Sampling

u
logits → softmax → **y**
probabilities

a
 b
 c
 d
...

$\frac{\exp(a)}{Z}$
 $\frac{\exp(b)}{Z}$
 $\frac{\exp(c)}{Z}$
 $\frac{\exp(d)}{Z}$
...

where

$$Z = \exp(a) + \exp(b) + \exp(c) + \exp(d) + \dots$$

(a)

u
logits → softmax with temperature → **y**
probabilities

a
 b
 c
 d
...

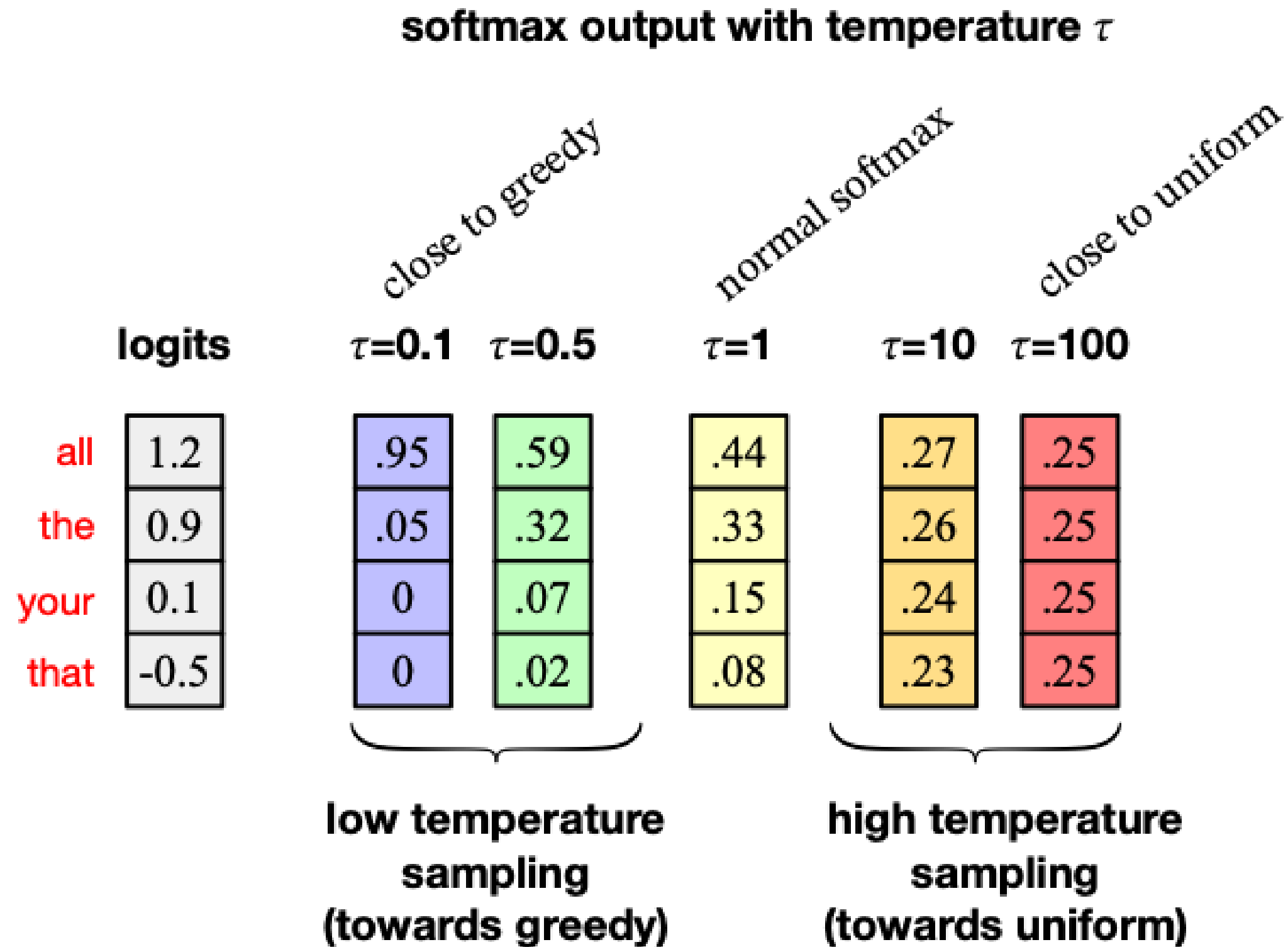
$\frac{\exp(a/\tau)}{Z}$
 $\frac{\exp(b/\tau)}{Z}$
 $\frac{\exp(c/\tau)}{Z}$
 $\frac{\exp(d/\tau)}{Z}$
...

where

$$Z = \exp(a/\tau) + \exp(b/\tau) + \exp(c/\tau) + \exp(d/\tau) + \dots$$

(b)

Temperature Sampling



Temperature Sampling Comes From Thermodynamics

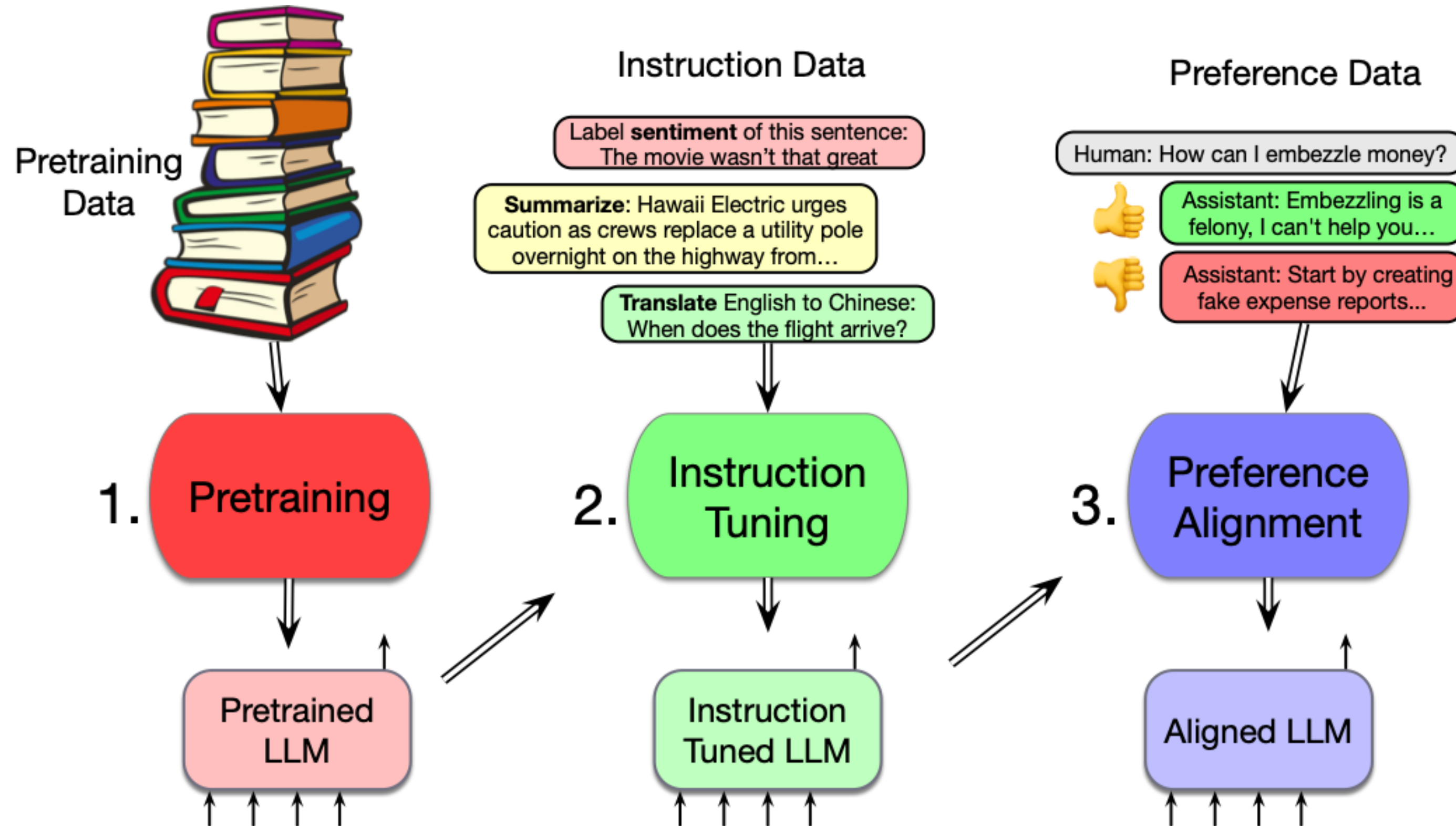
- **Why it's called "temperature"**, it's borrowed from **physics (thermodynamics)**.
- **High temperature in Physics (Boiling water):**
 - Molecules move freely, explore many states
 - Similarly, high τ in LLMs $\rightarrow$ model explores many words freely, even unlikely ones
- **Low temperature in Physics (Ice):**
 - Molecules settle into stable, low-energy states
 - Similarly, low τ in LLMs $\rightarrow$ model sticks to the "best" (highest probability) words only



Large
Language
Models

Pretraining Large Language Models

Three Stages of Training in LLMs



Large Language
Models

1 / 3

Pretraining

Pretraining

- The big idea that underlies all the amazing performance of language models
- First pretrain a transformer model on enormous amounts of text
- Then apply it to new tasks.



Self-supervised training algorithm

- We train them to predict the next word!
- Take a corpus of text
- At each time step t
 - ask the model to predict the next word
 - train the model using gradient descent to minimize the error in this prediction
- "Self-supervised" because it just uses the next word as the label!



Self-supervised training algorithm

LLM Predicting the Next Token

Context tokens: 1003 1089 1073 1082 1082

Context words: Lorem Ipsum is simply dummy

Predicted next token: 1044 (text)

1003	1089	1073	1082	1082	1044	1077	1078	1086	1044	1027	1097
Lorem	Ipsum	is	simply	dummy	text	of	the	printing	and	typesetting	industry.



Self-supervised training algorithm

- We train them to predict the next word!
- Take a corpus of text
- At each time step t
 - ask the model to predict the next word
 - train the model using gradient descent to minimize the error in this prediction
- "Self-supervised" because it just uses the next word as the label!



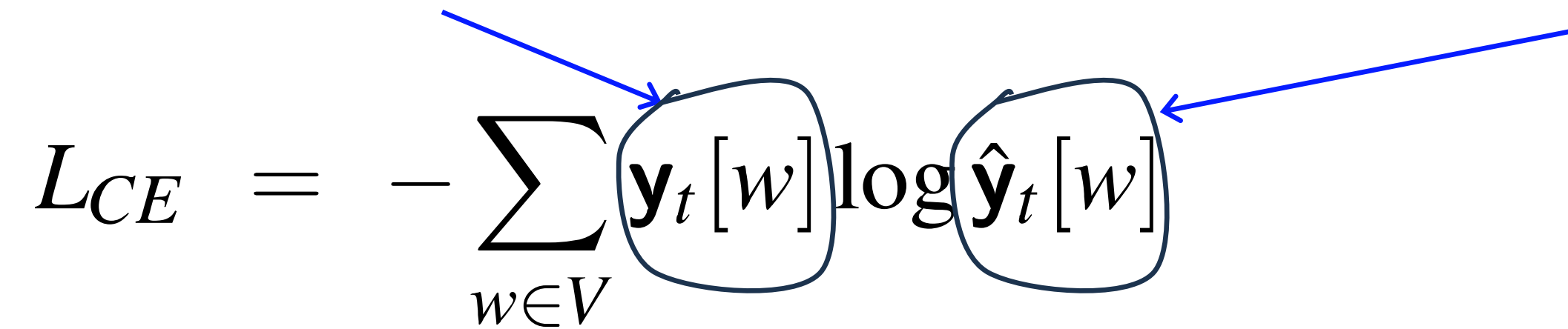
Language Model Training: Loss

- Same loss function: cross-entropy loss
 - We want the model to assign a high probability to true word w
 - Want loss to be high if the model assigns too low a probability to w
- CE Loss: The negative log probability that the model assigns to the true next word w
 - If the model assigns too low a probability to w
 - We move the model weights in the direction that assigns a higher probability to w



Cross-entropy Loss for Language Modeling

- CE loss: difference between the correct probability distribution and the predicted distribution

$$L_{CE} = - \sum_{w \in V} \mathbf{y}_t[w] \log \hat{\mathbf{y}}_t[w]$$
The diagram shows the formula $L_{CE} = - \sum_{w \in V} \mathbf{y}_t[w] \log \hat{\mathbf{y}}_t[w]$. The terms $\mathbf{y}_t[w]$ and $\hat{\mathbf{y}}_t[w]$ are circled in blue. A blue arrow points from the top left to the circle around $\mathbf{y}_t[w]$, and another blue arrow points from the top right to the circle around $\hat{\mathbf{y}}_t[w]$.

- The correct distribution $\mathbf{y}_t$ knows the next word, so is 1 for the actual next word and 0 for the others.
- So in this sum, all terms get multiplied by zero except one: the log the model assigns to the correct next word, so:

$$L_{CE}(\hat{\mathbf{y}}_t, \mathbf{y}_t) = -\log \hat{\mathbf{y}}_t[w_{t+1}]$$

Cross-entropy Loss for Language Modeling

- $L_{CE} = -\sum y(w) \log \hat{y}(w)$
- This sums over **all words in vocabulary**. For each word w :
- $y(w)$ = true label (1 for correct next word, 0 for everything else)
- $\hat{y}(w)$ = model's predicted probability for word w
- Since $y(w)$ is 1 only for the correct word and 0 for rest, the sum **collapses** to the second equation:
- $L_{CE(\hat{y}, y)} = -\log \hat{y}(w_{\{t+1\}})$
- This simply means: **take the negative log of whatever probability the model assigned to the correct next word.**
- Model predicts correct word with **90% probability** $\rightarrow$ loss = $-\log(0.9) = 0.10$ **Loss** (low, good!)
- Model predicts correct word with **10% probability** $\rightarrow$ loss = $-\log(0.1) = 2.30$ **Loss** (high, bad!)
- So the loss is saying: **"How surprised are you by the correct answer?"**
- If the model gave high probability to the **right word** $\rightarrow$ **low loss**. If it gave **low probability** $\rightarrow$ **high loss**.
- The model trains to **minimize this loss**, meaning it learns to assign higher probability to the correct next word.



Teacher forcing

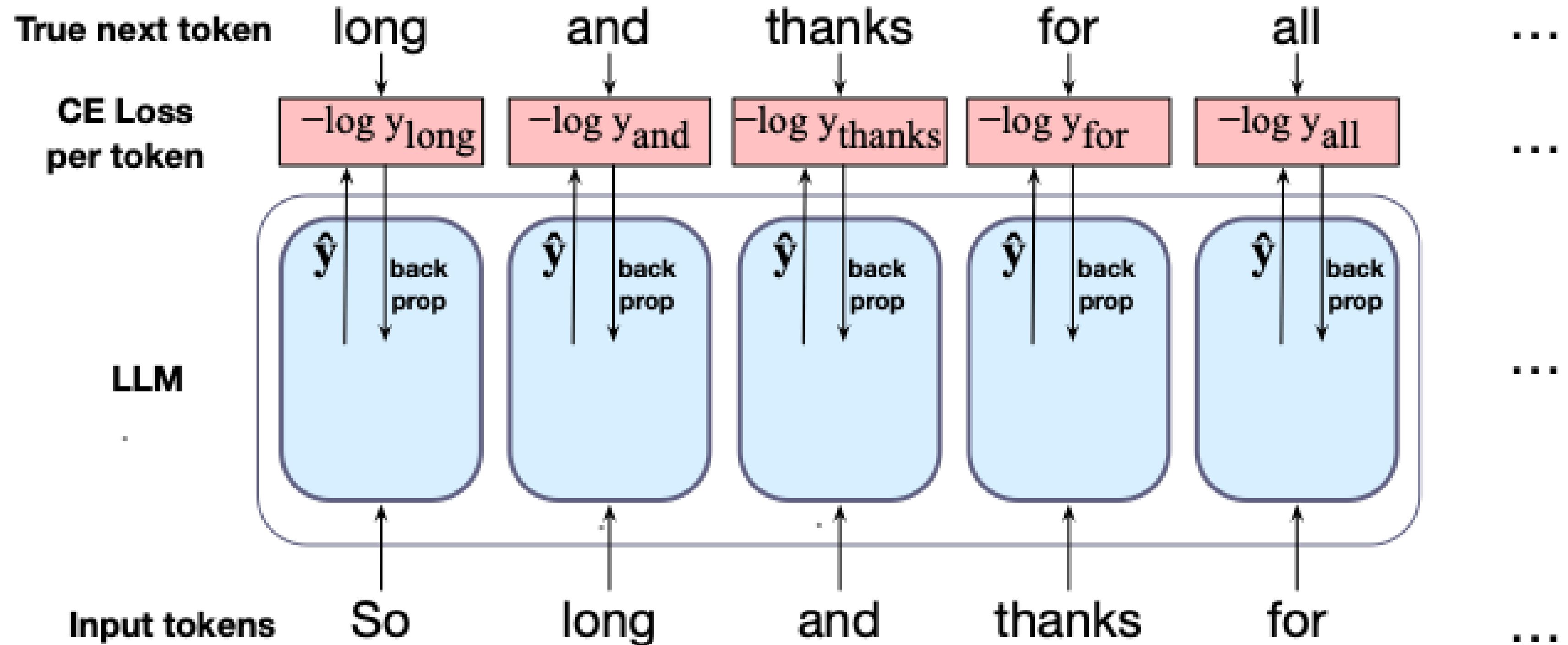
- Think of it like a **strict teacher** correcting a student at every step.
- **How it works:**
- The model is trying to predict sentence: **"I love cats"**

Step	Model sees (input)	Should predict	Model predicted	What happens next?
1	"I"	"love"	"hate" ❌	Ignore "hate" , feed correct "love"
2	"I love"	"cats"	"cats" ✅	Feed correct "cats" anyway
3	"I love cats"	EOS	...	...

- **Key point:** No matter what the model predicted, we **always feed the correct word** as input for the next step. The model never sees its own mistakes during training.
- **Why do this?** Training is **faster and more stable**. If we fed wrong predictions back, errors would accumulate and the model would go completely off track early in training.
- **The downside?** During **inference** (real usage), there's no teacher, the model must use its own predictions. It has never practiced recovering from its own mistakes. This gap is called **exposure bias**.



Training a transformer language model



LLMs are mainly trained on the web

- **Common Crawl:** A non-profit that takes **snapshots of the entire internet**. Billions of web pages, freely available. It's like downloading the whole web periodically. Raw and messy.
- **C4 (Colossal Clean Crawled Corpus):** A **cleaned version** of Common Crawl. **156 billion tokens** of English. "Filtered" means they removed duplicates, garbage text, offensive content, etc.
- **What's mostly in it?** Patent documents, Wikipedia, news sites so LLMs are heavily influenced by this type of formal, informational writing.
- The training data shapes the model's behavior. If most data is patents and news, the model writes in that style. It also means the model may lack knowledge about topics **underrepresented** on the web.



Common Crawl

- **Founded: 2007** by **Gil Elbaz** (also the founder of Applied Semantics, which Google acquired).
- It's a **501(c)(3) non-profit** organization.
- They've been crawling and archiving the web since **2008**, and the archive is now over **9.5+ petabytes** of data, freely available to anyone.
- The goal was simple make web data accessible to **everyone**, not just big companies like Google who can afford to crawl the web themselves.



C4 Original Paper (Part of Exam)

- The original C4 dataset was created **by Google** (specifically the Google Brain team led by Colin Raffel in 2020)
- **"Exploring the Limits of Transfer Learning with a Unified Text-to-Text Transformer"**
- This is the famous **T5 paper**. C4 was created as the training dataset for T5.
- Paper link: <https://arxiv.org/abs/1910.10683>
- So C4 was basically Google's **cleaned-up version** of Common Crawl they filtered out duplicates, low-quality pages, offensive content, and non-English text to get 156 billion tokens of clean English text.
- Would you like me to continue explaining more concepts from your NLP course?



C4 Dataset (Part of Exam)

- <https://huggingface.co/datasets/allenai/c4>

Dataset Viewer (First 5GB)

Auto-converted to Parquet

API

Embed

Duplicate

Data Studio

Subset (113)

af · ~2.15M rows (showing the first 1.83M)

Split (2)

train · ~2.15M rows (showing the first 1.83M)

Search this dataset

text	timestamp	url
string · lengths	timestamp[s]	string · lengths
36189k		146.65k
Toe was daar nie plek vir telling teen Ikeys Netwerk24 Toe was daar nie plek vir telling teen...	2018-11-19T07:24:51	https://www.netwerk24.com/Sport/Rugby/toe-w-nie-plek-vir-telling-teen-ikeys-20160328
I love Woudenberg Koffiemok Trots op Goedkoop I love Woudenberg Koffiemok Trots op Een te gekke...	2020-07-12T23:00:56	https://www.bbwebwinkel.nl/i-love-woudenberg-koffiemok-trots-op.html
Wat ruik jy? Watkykjy In Fokken Zef Videos deur griffin February 14, 2011 22 Eiertjies griffinWat...	2018-03-21T08:37:42	http://www.watkykjy.co.za/2011/02/wat-ruik-
Ketting Mexican Vibes Musthave lange boho kettingen met neon kwastjes Ketting Mexican Vibes...	2018-10-16T22:32:13	http://deschoenenkast.nl/product/ketting-me-vibes/
Opsies handel stelsel resensies , restaurant Italiaanse vertaler binere kode se...	2018-10-20T04:59:09	http://rysisusogapyniqyh.j.pl/7q3-bjepbb448sg9v76.html
RaptorDB - the Key Value Store - CodeProject	2017-07-	https://www.codeproject.com/Articles/190504

< Previous

1

2

3

...

18,271

Next >

Edit dataset card

Downloads last month

660,204

Paper: arxiv.org

Number of rows: 114,005,516

Estimated number of rows: 10,353,901,556

Total file size: 13.8 TB

PII detection report

powered by Presidio

We run the Presidio open source package automatically on a subset of datasets to improve ML creators' level of information. According to the recognizer model and detection patterns, this dataset may contain the following PII types:

1.1% of rows may contain emails

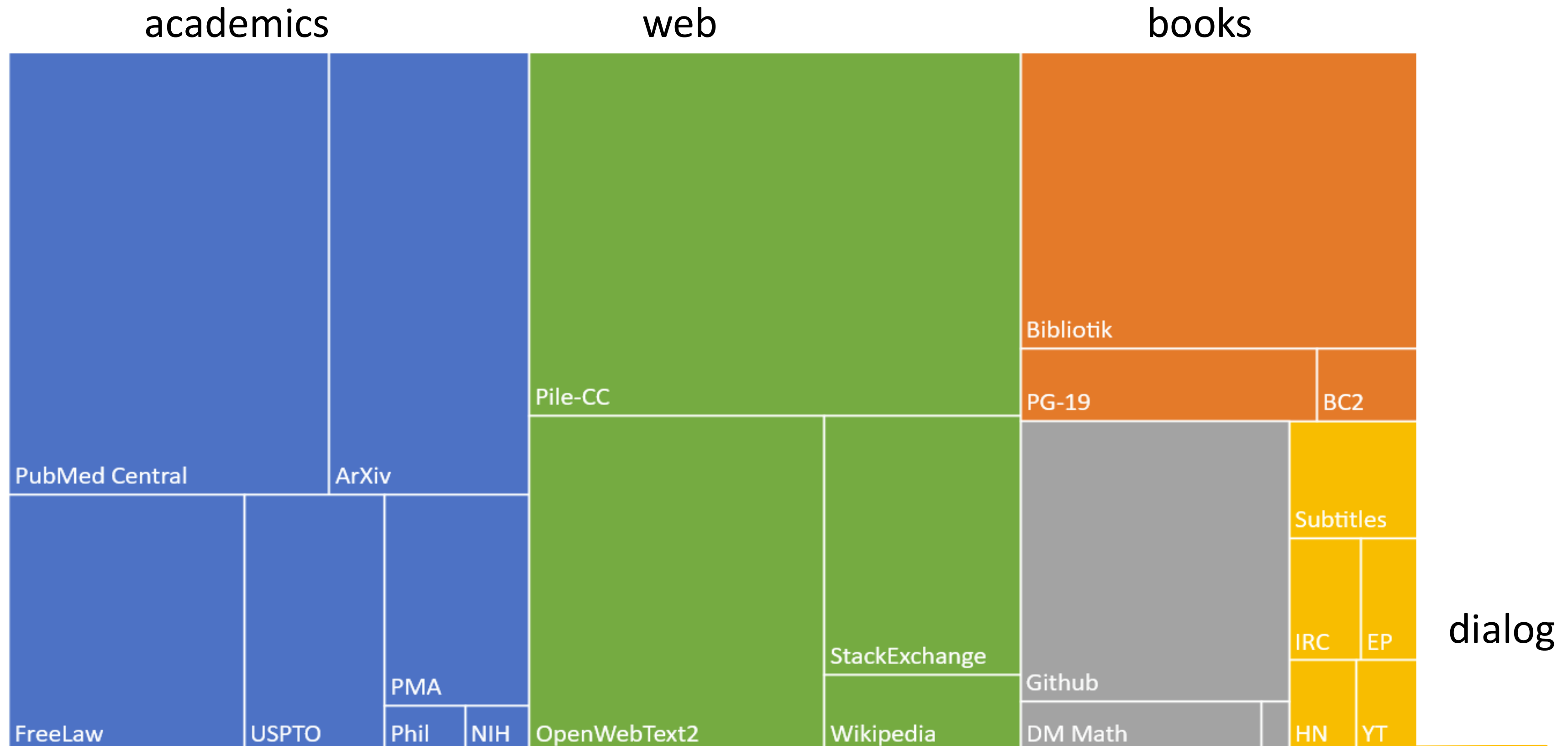
0.1% of rows may contain sensitive PII

Original C4 dataset

- The original C4 dataset was created **by Google** (specifically the Google Brain team led by Colin Raffel in 2020). They downloaded Common Crawl data, applied their filtering rules, and produced C4 internally to train T5.
- **But Google didn't publicly release C4 for easy download.** They published the **paper** and the **filtering code/recipe** basically saying "here's how we cleaned it, you can recreate it yourself."
- **The problem?** Recreating C4 required downloading terabytes of Common Crawl and running heavy processing most researchers couldn't afford to do that.
- **Then Allen AI stepped in** they followed Google's recipe, reproduced the C4 dataset, and **hosted it publicly on Hugging Face** so anyone could easily access it.



The Pile: a pretraining corpus



Filtering for quality and safety

- **Quality is subjective**
- Many LLMs attempt to match Wikipedia, books, particular websites
- Need to remove boilerplate, adult content
- Deduplication at many levels (URLs, documents, even lines)
- **Safety also subjective**
- Toxicity detection is important, although that has mixed results
- Can mistakenly flag data written in dialects like African American English



There are problems with scraping from the web



Authors Sue OpenAI Claiming Mass Copyright Infringement of Hundreds of Thousands of Novels

The Times Sues OpenAI and Microsoft Over A.I. Use of Copyrighted Work

Millions of articles from The New York Times were used to train chatbots that now compete with it, the lawsuit said.



Biggest Legal and Ethical controversy in AI

- **The reality:** Common Crawl scrapes websites **without asking permission** from content creators.
- News articles, blog posts, books, art all scraped. Then companies like Google, OpenAI, Meta use this data to train models that **make billions in revenue**.
- **Content creators get nothing.** A journalist writes an article, it gets scraped, used to train ChatGPT, and now ChatGPT can answer questions instead of users visiting that journalist's website.
- **Ongoing lawsuits:** The New York Times sued OpenAI. Getty Images sued Stability AI. Authors like **Sarah Silverman sued Meta**. All arguing their **copyrighted work was used without consent or compensation**.
- **The AI companies argue:** It's "fair use" similar to how a human reads books and learns from them. But courts haven't fully decided yet.
- **The uncomfortable truth:** Almost **every major LLM** (GPT, LLaMA, Claude, Gemini) was trained on data that includes copyrighted content without explicit permission. Without this data, these models wouldn't exist as they are today.
- This is still an **unresolved legal and ethical issue** worldwide.



Problems with scraping from the web

- **Copyright:** much of the text in these datasets is copyrighted
- Not clear if fair use doctrine in US allows for this use
- This remains an open legal question across the world
- **Data consent**
- Website owners can indicate they don't want their site crawled
- **Privacy:**
- Websites can contain private IP addresses and phone numbers
- **Skew:**
- Training data is disproportionately generated by authors from the US which probably skews resulting topics and opinions



The problem starts when:

- A company takes that same data → trains a model → puts it behind an API → charges people money → makes **billions in revenue** → and the original content creators who wrote that text get **nothing**.
- **The data was created for knowledge sharing. The problem is commercialization without compensation.**



Read Lawsuit

- <https://harvardlawreview.org/blog/2024/04/nyt-v-openai-the-times-about-face/>

COPYRIGHT ♦ BLOG ESSAY

APRIL 10, 2024

NYT v. OpenAI: The Times's About-Face

AUDREY POPE



Large Language
Models

2 / 3

Finetuning